DIGITAL TWINS · PROGRESSIVE FIDELITY

A Layered Architecture for Digital Twins with Progressive Fidelity

How a digital twin can grow in fidelity, without rewriting what already works.

Arthur Chevallier · German Castignani

Luxembourg Institute of Science and Technology

Workshop talk · 15 minutes

Open-source prototype · EVSK · VirtualSky International Airport

DEFINITION

Fidelity in a digital twin

Fidelity describes how closely a digital twin represents the system it models.

Spatial

geometry and position

Logical connections
↓
Coarse geometry
↓
Detailed geometry

Temporal

timing and change

Coarse event timing
↓
Finer time resolution
↓
Subsecond updates

Behavioural

how the system responds

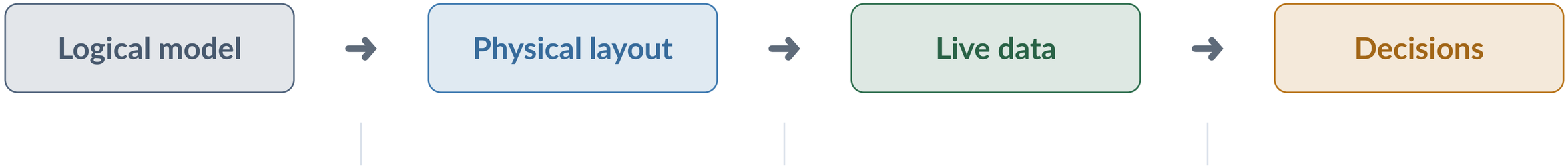
State transitions
↓
Rules and interactions
↓
Data-calibrated dynamics

Fidelity depends on the purpose. A twin can be detailed in one dimension and simplified in another.

THE PROBLEM

Digital twins are not born complete, they grow

When contracts change, a fidelity upgrade can break what already works.



Entity IDs disappear

A geometry upgrade replaces entity IDs with coordinates. Existing lookup logic loses its inputs.

Live feeds disagree

A feed uses different entity IDs or timestamp conventions. Objects duplicate and state estimates drift.

Decisions bypass events

An optimiser changes state without emitting events. The audit trail and replay no longer match.

The cost spreads beyond the new layer: consumer rewrites, repeated validation, and past experiments that may no longer be reproducible.

The gap in existing frameworks

What they give us

- ISO 23247: manufacturing reference architecture
- Tao et al.: physical / virtual / data / services / connections
- Redelinghuys et al.: acquisition to emulation

Good decompositions of **what a twin is made of.**

What they leave open

None make **fidelity-preserving evolution** an explicit constraint. Higher fidelity should **extend** a lower layer, not restructure it.

> **RQ1** How can a digital-twin architecture support progressive increases in fidelity **while preserving the contracts and functionality of previously implemented layers?**

THE CORE IDEA

Raise fidelity by **extending** a layer's contract, never by rewriting it.

Each layer L_i exposes a contract C_i. Upgrading may **extend** C_i, but must not modify or remove any existing element of any C_j. Append-only contracts: the layers above keep running, untouched.

EVSK · VirtualSky International Airport

An open-source airport twin for studying how fidelity grows.

Why an airport?

Flights, passengers, baggage and weather interact across different timescales.

Established delay, cost and flow models let us focus on **the architecture and its boundaries**.

Synthetic operations. Optional live feeds at L2. Open-source and DOI-archived.

Four implemented layers

L0 **Logical core**
Domain services, Kafka events and Neo4j

L1 **Physical layout**
Gates, routes and travel times

L2 **Observed data**
Geospatial context and switchable feeds

L3 **Decision support**
Analysis, costs and gated recommendations

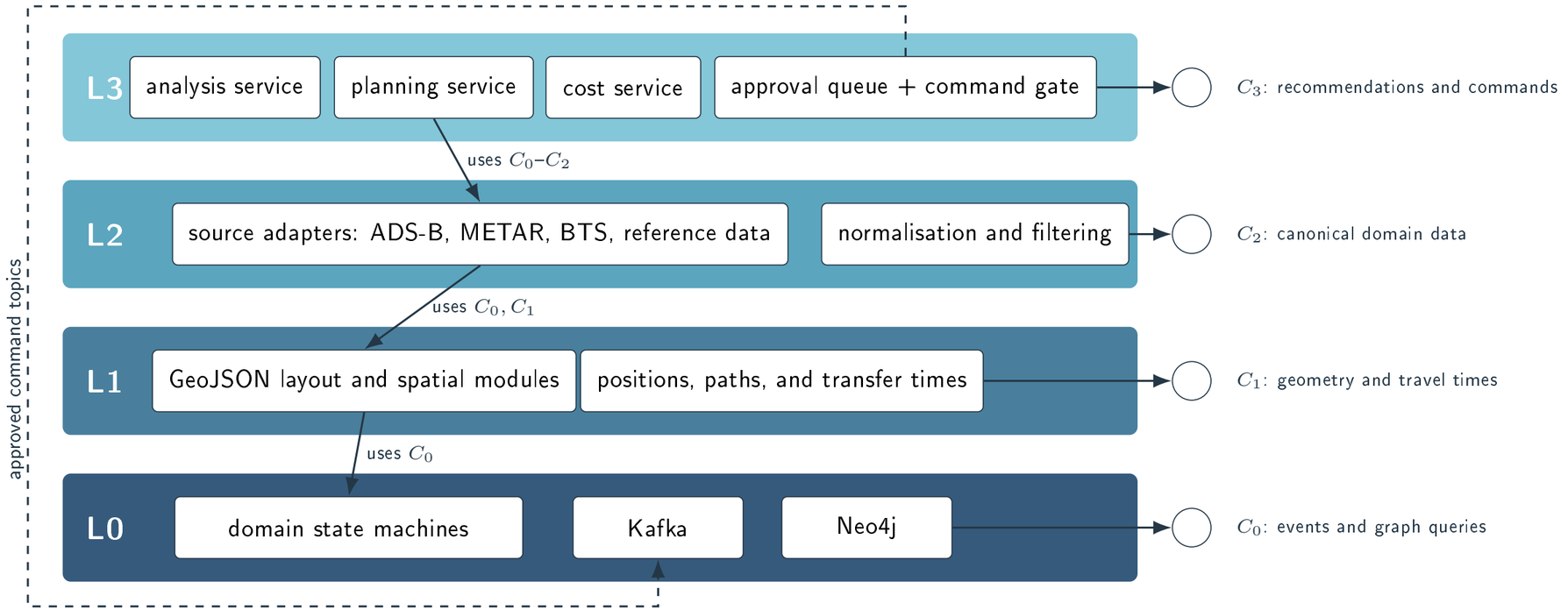
01

A four-layer fidelity architecture

Explicit contracts · append-only evolution · a decisional layer that closes the loop.

Four capability layers

L0 L1 L2 L3



Solid arrows denote contract dependencies; the dashed arrow is the approval-gated command path. No domain service calls another directly.

L0 logical event-driven core (Kafka + Neo4j) · **L1** physical layout & travel times · **L2** geospatial + real-world feeds (ADS-B, METAR, BTS) · **L3** decisional, actions return only through approval gates.

How the loop closes

A layer depends on the **contracts** of lower layers, **never on their internals.**



← approved actions re-enter only through **command topics** (never through imports) ←

Because L3 re-enters only through command topics, it stays **removable** from the shadow below.

02

EVSK: the twin, actually running

VirtualSky International Airport · an open-source, ongoing research prototype.

One instrument, four layers

9	420	~30k	3600×
DOMAIN SERVICES	FLIGHTS / SIM DAY	PASSENGERS / DAY	MAX SIM CLOCK

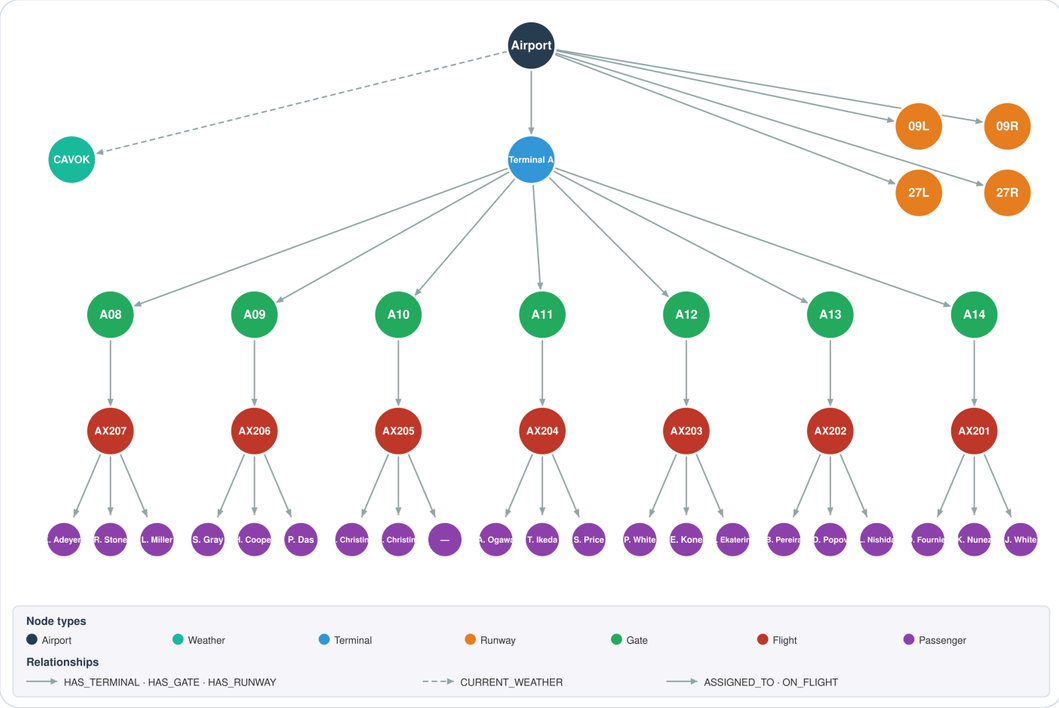
The stack

- 9 FastAPI domain services + Node/Express gateway
- Kafka event backbone
- Neo4j graph as the single source of truth
- React / TypeScript dashboards
- LightGBM queue forecasting
- Config-driven airport identity; Docker Compose; staged CI

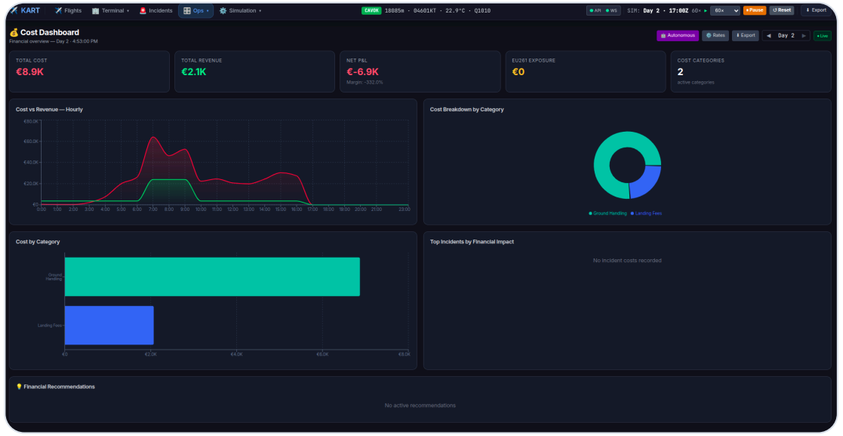
The decisional layer, for real

- Anomaly detection, scenario & delay-cascade analysis
- Cost / P&L, EC261 exposure, carbon, Monte-Carlo NPV and IRR
- Gated recommendations: human-in-the-loop commands

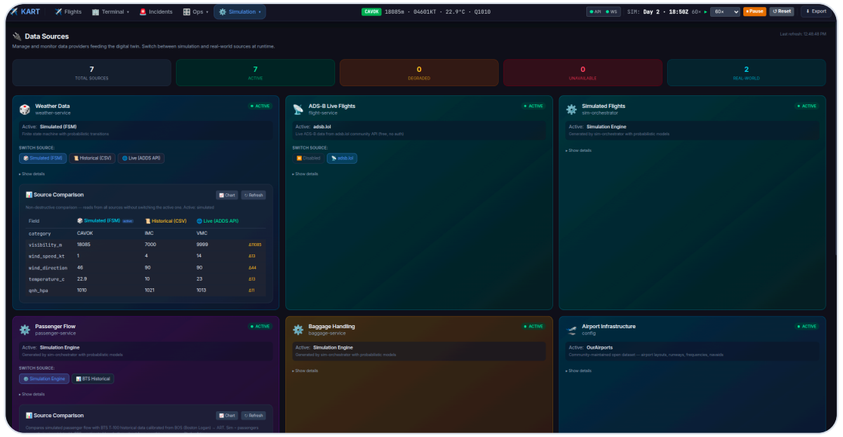
The twin, live



L0 · Neo4j graph, the single source of truth: airport, terminals, gates, runways, flights, passengers, weather.



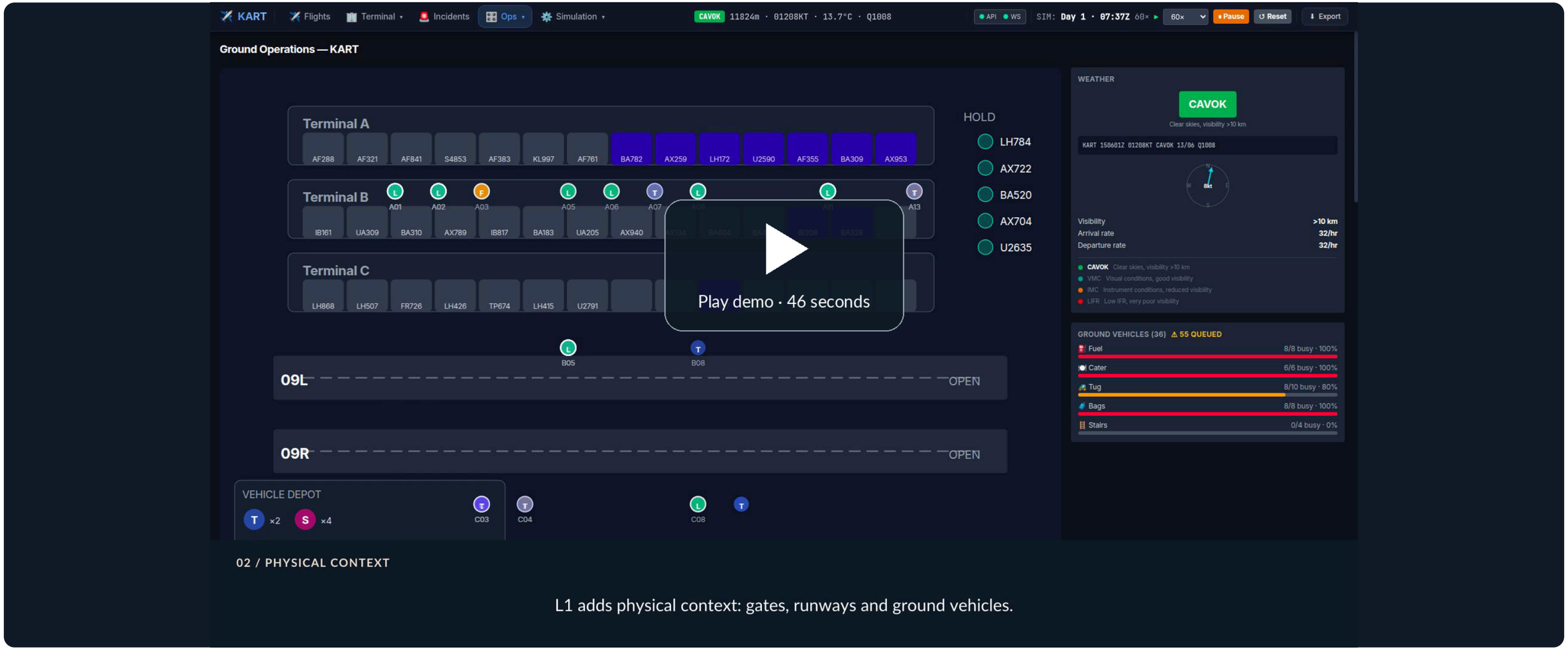
L3 · cost / P&L dashboard.



L2 · swap simulated and live feeds at runtime.

RECORDED DEMONSTRATION

The twin, running



46-second recording · synthetic operations · English subtitles

Keeping contracts honest: specification-first

RQ2

EVSK was built with AI-assisted, specification-first development.

Service specifications and skill files guided the AI coding agents before implementation.

RQ2 To what extent does specification-first development **preserve architectural contracts** during sustained implementation, and **where does it need independent verification?**

Specs are authoritative

- event schemas
- Neo4j data model
- SimClock rules
- `SPEC.md` per service
- skill files

Guardrails

- no wall-clock in logic
- no service-to-service HTTP
- no silent mutations
- graph = source of truth
- contract & fidelity tests

But specs alone were not enough. GAP A mid-project audit found constants that had duplicated **and diverged**, recommendations never wired to command topics, and an ML endpoint that reported success **without training a model.** Each fix went back into the skill files.

03

Does the principle actually hold?

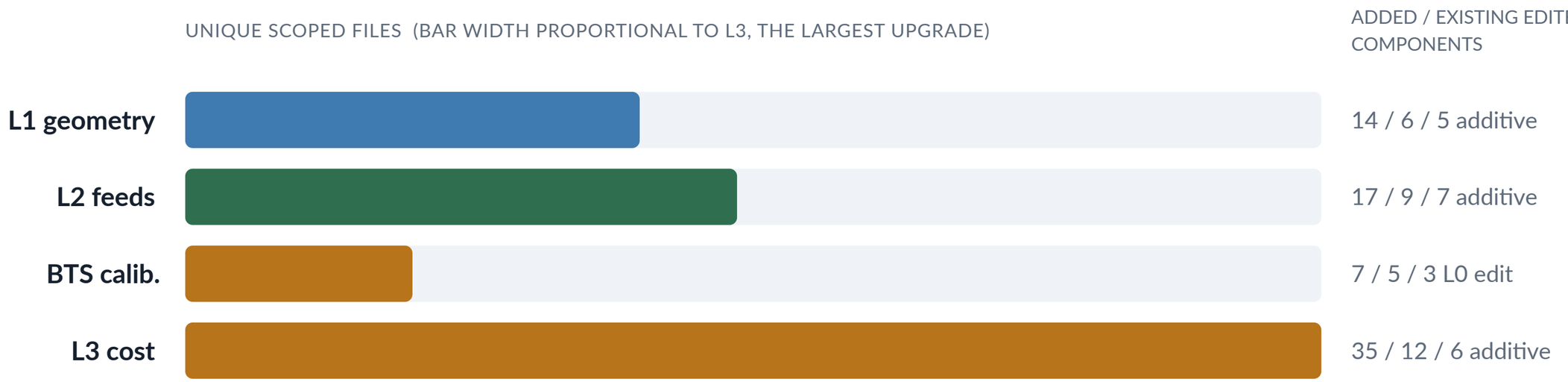
A retrospective audit of four real fidelity upgrades in the repository history.

EVIDENCE

Blast radius of four fidelity upgrades

RQ1

3 of 4 upgrades touched only new code. No existing consumer needed an interface migration.



BTS calibration: why it is different. L0 exposed no input port for calibrated parameters. Calibration therefore had to edit the schedule generator and passenger generator directly. **Contracts stayed stable; internals did not.** The fix: adding a dedicated calibration input port to L0.

What we found

FINDING

RQ1 · contract preservation

- All four paths preserved existing event & service contracts.
- **No** existing consumer needed an interface migration.
- Supports **append-only** contracts: preservation of the contract, not a promise of zero lower-layer edits (see BTS).

RQ2 · specification-first

- Specs, skill files and hard rules **help**, but are **not** sufficient alone.
- Contract tests, fidelity regressions and audits are necessary complements.
- Architecture says what must stay stable; methodology helps preserve it.

Perspectives and next steps

Open questions

- When does a contract **need** to be versioned rather than extended?
- Can contract preservation be checked automatically at CI time, without a manual audit?
- How does the architecture behave in non-event-driven stacks (REST-only, streaming)?

Planned work

- **Compare architectures**: run the same upgrades on monolithic and other layered twins. Measure changed files, contract breaks and implementation effort.
- Test generality in a second domain, such as a port or hospital
- Formalise contracts for static or property-based checks
- Audit the next fidelity upgrade as it happens

EVSK is active and open. Contributions and replication studies welcome. github.com/arthur-chevallier/airport-digital-twin

TAKEAWAY

Treat fidelity growth as a first-class architectural constraint, and verify it.

The same ordering generalises far beyond airports: event state, space, observed data, decisions.

Thank you

Questions welcome.

github.com/arthur-chevallier/airport-digital-twin

A Layered Architecture for Digital Twins with Progressive Fidelity

Arthur Chevallier · German Castignani · Luxembourg Institute of Science and Technology

EVSK is open source · all data synthetic except opt-in L2 live feeds